Serial Decode for the Keithley DMM6500 — notes for beta testers

version 1.56 · `Serial_Decode.tspa` · a TSP app that turns a DMM6500 into a UART/LIN decoder

It digitizes a serial line, finds the bit rate and framing itself, and shows the decoded bytes on the front panel. No host computer is needed once it is loaded.

Everything below is either a documented behaviour that **looks** like a bug, or a defect I already know about. Please read it before reporting anything — it will save you rediscovering my own scars.

Loading it

Copy `Serial_Decode.tspa` to a USB key, insert it, and load it from **Menu** → **Scripts**. Run the script; it builds its own screens.

Relaunch with *End App*, not by reloading from the host. The firmware never reclaims display object IDs, so there is a hard limit on how many times the UI can be built between power cycles — about **4** host reloads, but about **12** if you End App and relaunch the script. When it runs out the screen simply stops being drawn, which looks exactly like a broken script. A power cycle resets it.

Four behaviours that are correct, or known, and not worth reporting

A signal whose voltage band straddles ground decodes inverted — deliberately. If the low level is below −1 V and the high level above +1 V, the app reads the line as RS-232 at line levels, which marks at its *negative* level. Feed it a single-supply logic waveform that has been DC-shifted across 0 V and every byte comes back wrong, self-consistently, with the right bit rate and framing. This cost me weeks and two false bug reports. **Keep the whole band on one side of ground** unless you actually mean RS-232.

A bit rate printed with a trailing `?` — e.g. `38400?` — means the app snapped the measurement to a standard rate but does not stand behind it. It locks to the rate it *measured*, not the one displayed. That is a deliberate hedge, not a rendering fault.

125000 Bd can be reported as 250000. A real defect, and I have localised it: the bit-time fit is correct and a later rescaling step halves it. Similarly 37500 can read as 38400, and 125000 as 128000, when the fit lands slightly high. Known, characterised, unfixed on purpose.

7E1 traffic is read as 8N1. The parity bit lands in bit 7, so every byte with odd parity comes back `+0x80` and roughly half the payload looks corrupt. Known.

Logging

The app writes logs to `/usb1/SERDEC`. With no key inserted, or a full one, logging silently does nothing — if you expected a file and got none, check the key first. Capture and decode still work in every mode without a key; the recordings put their bytes on the panel and say `NOT recorded`.

The key can be pulled and replaced while the app runs. The slot is polled twice a second while idle, so the buttons and the log cell follow it within half a second, and a replacement key gets a new name.

Screenshots are better than photographs

The DMM6500 will hand you a pixel-exact 800×480 capture of its own screen over its LXI web interface, with nothing loaded and no script running. It is undocumented by Keithley — I read it out of the instrument's own virtual-front-panel JavaScript:

```
POST http://<ip>/ajax_proc      function=9              -> session=<id>
GET  http://<ip>/images/fp.tga?<session>:<milliseconds>  -> uncompressed TGA, 800x480
GET  http://<ip>/images/fplow.tga?<session>:<ms>         -> half-res 400x240
```

HTTP basic auth, factory default `admin` / `admin`. `tools/screenshot.py` in the repository does this and writes a PNG; change `HOST` to your instrument's address.

Please send the whole frame rather than a crop. The numbers that diagnose most problems live in the header and footer lines, and a screenshot shows what the app *drew* — which is not always what it computed. Several of this project's bugs lived exactly in that gap.

Reporting something

Please open an issue:

https://github.com/ameline-design/dmm6500-serial-decode/issues

Include, as far as you can:

1. **A screen grab** — the whole 800×480 frame, per the section above. If the panel changed as you watched, one before and one after is worth more than either alone.
2. **The text that should have been decoded.** Paste the exact bytes or string you were transmitting. This is the single most valuable thing in the report: it turns "the decode looks wrong" into a difference I can compute, and it is the one piece I cannot reconstruct from anything else.
3. **The bit rate you set**, as a number, and whether it is a standard rate or something unusual.
4. **The framing you sent** — data bits, parity, stop bits, and the idle polarity if you know it (8N1, 7E1, 8E2 …). If the app reported a framing different from the one you sent, say both.
5. **The signal levels** — low and high in volts, and whether the band crosses ground. Read the first behaviour above before deciding this does not matter; it is the most common cause of a decode where every byte is wrong and every other number is right.
6. **Anything that needed a power cycle** to recover, and what you had just done.

A rough report is far better than none — send what you have and I will ask.

A wrong number that looks plausible is worth more to me than a crash. Crashes I can find on my own; confidently-wrong answers are what this project exists to hunt, and they are exactly what an outside pair of eyes is best placed to notice. I have measured this from the inside for months, so the gaps are unlikely to be in the arithmetic — they are much more likely to be in whether any of it makes sense to somebody who did not build it. A panel number you cannot interpret, or two that seem to disagree, is a real finding, not a naive question.